

Demystifying Containers



This is the second part of the two-part series where we continue to uncover the magic of container technology. The first part was published in the September 2020 issue of *Open Source For You*.

In the first part, we briefly went over what containers are and the reason why they came into existence, by discussing the problem statement that they aimed to solve. We started out on building a container from scratch. During the process, we covered namespaces and the important part that they play in container technology, and now, we shall be continuing from where we left off.

Having made the necessary changes to the code as discussed in the last part, if we now run our script as it is, we will notice that the results that we get are still not quite different. To understand why, we need to know what exactly happens behind the scenes when we run the `ps` command.

It turns out that `ps` looks at `/proc` directory to find out what processes are currently running on the host. If

we run `ls /proc` from our host OS and then from our container, we can see that the contents of the `/proc` directory in both cases are one and the same. To overcome this hurdle, we want the `ps` of our container to be looking at a `/proc` directory of its own. In other words, we need to provide our container its own file system. This brings us to an important concept of containers: layered file systems.

Layered file systems

Layered file systems are how we can efficiently move whole machine

images around. They are the reason why the ship does not sink. At a basic level, layered file systems optimise the call to create a copy of the root file system for each container. There are numerous ways of doing this.

Btrfs (<https://btrfs.wiki.kernel.org/>) uses copy on write (COW) at the file system layer. Aufs (<https://en.wikipedia.org/wiki/Aufs>) uses union mounts. Since there are so many ways to achieve this step, we will just use something simple. We will make a copy of the file system. It is slow, but it works.

```
root@ubuntu-vm:/home/ubuntu/Projects/make-sense-of-containers# ls /
bin  dev  home  lib  lib64  lost+found  mnt  proc  run  snap  sys  usr
boot  etc  HOST_FS  lib32  libx32  media  opt  root  sbin  srv  tmp  var
root@ubuntu-vm:/home/ubuntu/Projects/make-sense-of-containers# ls ubuntu-fs/
bin  CONTAINER_FS  etc  lib  lib64  media  opt  root  sbin  srv  tmp  var
boot  dev  home  lib32  libx32  mnt  proc  run  snap  sys  usr
root@ubuntu-vm:/home/ubuntu/Projects/make-sense-of-containers#
```

Figure 1: Listing the root directory of the host, and root that is going to be mounted in our container

